50. systemd Service Hardening Directives

50. systemd Service Hardening Directives

Phase: Phase 7 — Services and boot
Estimated time: 1–2 days
Slug: `50-systemd-hardening`

Learning objectives

- Apply ProtectSystem/NoNewPrivileges ideas
- Constrain WritablePaths
- Run as dedicated DynamicUser/User
- Read systemd-analyze security

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

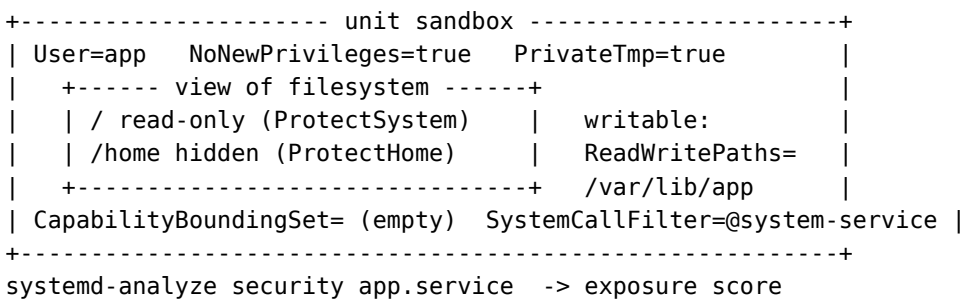
Why this matters

Every network service is one bug away from being controlled by someone else. When that happens, the question is how much damage the compromised process can do. A service running as root with full filesystem access can read your SSH keys, install a backdoor, and pivot to the next host. The same bug in a service that is sandboxed by systemd yields a process that can barely touch anything. Hardening directives cost a few lines in a unit file and give defence in depth without changing the application.

The single most valuable line to remember is `NoNewPrivileges=true`. It prevents the service and all its children from gaining more privileges via setuid binaries, file capabilities, or similar tricks. Even if an attacker runs `sudo` or a setuid helper, escalation fails.

Core concepts

Directive	Effect
`User=` / `DynamicUser=yes`	run as unprivileged account, not root
`NoNewPrivileges=true`	block privilege gain through exec
`PrivateTmp=true`	private `/tmp` and `/var/tmp` for the unit
`ProtectSystem=strict`	whole file system read-only except listed paths
`ProtectHome=true`	`/home`, `/root` hidden
`ReadWritePaths=`	explicit writable exceptions
`CapabilityBoundingSet=`	limit Linux capabilities
`PrivateDevices=true`	no access to real devices
`SystemCallFilter=`	allow list of syscalls
`RestrictAddressFamilies=`	limit socket types



Worked examples

Example 1: a hardened unit for a small web app.

```
[Unit]
Description=Example app
After=network.target

[Service]
User=myapp
Group=myapp
ExecStart=/opt/myapp/bin/server --port 8080
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/var/lib/myapp
```

```
PrivateDevices=true
ProtectKernelTunables=true
ProtectControlGroups=true
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX
CapabilityBoundingSet=
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Example 2: measure exposure and read the report.

```
systemd-analyze security myapp.service | head -20
systemd-analyze security myapp.service --no-pager | tail -3
```

Lower exposure is better; treat the score as guidance, not a goal in itself.

Example 3: a drop-in instead of editing the vendor unit.

```
sudo systemctl edit nginx.service
# in the editor:
# [Service]
# NoNewPrivileges=true
# ProtectHome=true
sudo systemctl daemon-reload && sudo systemctl restart nginx
```

Example 4: find why a hardened service fails, e.g. it cannot write a file.

```
journalctl -u myapp.service -n 30 --no-pager
sudo systemd-run --pty -p ProtectSystem=strict -p ReadWritePaths=/var/lib/myapp
bash
```

A “Read-only file system” error means a needed path is missing from `ReadWritePaths=`.

Example 5: confirm the setting is live.

```
systemctl show myapp.service -p NoNewPrivileges -p ProtectSystem -p User
```

Mini-lab

1. Create a user `myapp` and a unit running `python3 -m http.server 8080` from `/srv/site`.
2. Run `systemd-analyze security myapp.service` and note the score.
3. Add `NoNewPrivileges=true`, `PrivateTmp=true`, `ProtectHome=true`. Re-run the analysis; compare.
4. Add `ProtectSystem=strict` and see what breaks. Fix it with `ReadWritePaths=` only for what the app writes.
5. Run `ls /tmp` from inside the service via a drop-in test command to see the private tmp.
6. Record each directive and the specific attack it blunts.

Decision guide / failure modes

Need	Directive
Cannot become root again	`NoNewPrivileges=true`
Stop reading user data	`ProtectHome=true`
Limit write damage	`ProtectSystem=strict` + `ReadWritePaths=`
No raw sockets or exotic nets	`RestrictAddressFamilies=`
Bind port below 1024 without root	`AmbientCapabilities=CAP_NET_BIND_SERVICE`

Failure modes: applying everything at once so nothing works and nobody knows which line broke it; `ProtectSystem=strict` without testing writable paths; the app genuinely needing setuid (some mail daemons), in which case record the exception; relying on the analyze score instead of reading directives; and skipping a restart test after reboot. Add one directive at a time.

Common mistakes

- Running network services as root without need
- ProtectSystem without testing paths
- Ignoring analyze security suggestions forever

Exercises

Work in `exercises/50-systemd-hardening/`. Read that folder's `README.md` fully before coding.

```
cd exercises/50-systemd-hardening
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `nonewprivileges_directive`. It prints the exact systemd unit-file line, including the directive name, an equals sign, and the value, that stops a service and its children from gaining privileges through setuid or capabilities. Case matters because systemd directives are case sensitive. Print one line, nothing else. After it passes, apply the directive to a throwaway unit in your lab VM and verify with `systemctl show UNIT -p NoNewPrivileges`. Record the before and after exposure from `systemd-analyze security` and list one thing that stopped working when you added `ProtectSystem=strict`.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes

- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/50-systemd-hardening/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. NoNewPrivileges meaning?
2. PrivateTmp purpose?
3. Why dedicated User?

Further reading

- https://www.freedesktop.org/software/systemd/man/systemd.exec.html